QA Process

APPN Aerial SOP — Locked revision 1.03

APPN – Aerial Data QC Protocols

Important

The full QC pipeline is still under active development. It is being developed and tested as part of the early-season APEx flights, and parts of this protocol may change as that work progresses.

Note

This document describes procedures for measuring the uncertainty of drone flights. Initially this document focuses on the hyperspectral drones, but the procedure will expand in the future. It will also form the basis of a standard QC procedure for future flights.

Note

The QC procedures in this document are tightly coupled to the APPN flight designs — the panels, ground control points, and reflectance targets that the QC steps rely on are deployed *during* the flight itself. Before performing any QC, make sure you understand which targets were placed in the field by reading the relevant flight protocol:

- Standard Flight — routine APPN data-collection flight; defines the baseline panel and GCP layout used for ELM and positional QC.
- Validation Flight — extended flight that adds additional validation panels, GCPs, and (in future) LiDAR calibration targets used by the spectral, positional, and LiDAR QC sections below.

Document Structure

The conventions in General QC Conventions apply to **all** QC processes below. Read that section first, then jump to the specific QC type you need:

- General QC Conventions
- File Format — GeoJSON vs Shapefile
- Naming Conventions
- File Storage
- Positional QC
- Spectral QC
- Creating Geospatial Vector Data — GOBI & CALViS (QGIS)
- Extracting Pixels into a Table
- Comparing Runs (multi-run QC)
- LiDAR QC

General QC Conventions

These file-format, naming, and storage rules apply to every QC process (positional, spectral, LiDAR) described later in this document.

File Format — GeoJSON vs Shapefile

Important

GeoJSON (`.geojson`) is the preferred format for all QC vector data, though shapefiles (`.shp`) are also supported by the QA and plot extraction code https://github.com/ArdenB/APPN_GenricFileStorage.

GeoJSON is preferred because:

- It is a **single self-contained file**, rather than the 4–6 sidecar files (`.shp`, `.shx`, `.dbf`, `.prj`, `.cpg`, …) required by the shapefile format. This makes it easier to copy, share, and store without losing pieces.
- It is **plain-text JSON**, so it is human-readable, diff-friendly, and works cleanly with Git version control.
- It is an **open, web-native standard** (RFC 7946) supported by virtually all modern GIS tools, web mappers, and Python/R libraries.
- CRS handling is explicit: the default GeoJSON specification (RFC 7946) only supports WGS84 (EPSG:4326), but all major GIS software (QGIS, ArcGIS, GDAL, geopandas, etc.) allow you to specify and read a different CRS when writing/reading the file. For APPN QC work we **keep the file in the same projected CRS as the source orthomosaic** (e.g. GDA2020 / MGA zone XX) rather than reprojecting to WGS84.
- It has **no field-name length limit** (shapefile DBF caps at 10 characters) and no 2 GB file-size cap.

If you already have shapefiles, you do **not** need to re-create them — the QA pipeline will read either format. New files should be saved as `.geojson`.

Naming Conventions

Tip

The exact file names to use for the most common equipment setup are listed in the worked example below the table.

All QC vector files follow a single pattern:

`QC_{TargetType}_{TargetIdentifier}_{Descriptor}.geojson`

Where:

- `QC` — fixed prefix that flags the file as a QC vector layer so it is picked up by the QA pipeline.
- `{TargetType}` — what kind of QC the file supports. Current values are `ELM` (reflectance panels used to build the ELM), `VAL` (validation panels/targets *not* used in the ELM), `GCP` (ground control points for positional QC), and `LIDAR` (LiDAR calibration surfaces).
- `{TargetIdentifier}` — **optional.** Omit this segment when there is only a single standard target of that type in the flight. Add it when there are multiple targets, or when downstream reporting needs to distinguish between them (see the per-row rules below). A few identifiers are **reserved** and always carry a fixed meaning — most importantly `groundtruth` on a `GCP` file, which marks the surveyed reference layer (see the `QC_GCP_groundtruth_points.geojson` row).
- `{Descriptor}` — what the geometry represents (e.g. `Panels`, `points`, `Surface`).

The specific names in use today are:

Name	Overview
`QC_ELM_Panels.geojson`	Polygons of the reflectance panels used in the ELM during GRYFN Processing. Use this exact name when a **single** GRYFN 4-panel set was used for the ELM. If two 4-panel sets are used together for the ELM, add a target identifier for each (e.g. `QC_ELM_east_Panels.geojson` / `QC_ELM_west_Panels.geojson`, or an inventory tag such as `QC_ELM_blue_Panels.geojson` / `QC_ELM_yellow_Panels.geojson` — USYD, for instance, tags its two sets blue and yellow).
`QC_VAL_Panels.geojson`	Reserved for the **GRYFN dedicated 2-panel validation set**. Each node has exactly one of these, so no target identifier is needed.

Name	Overview
QC_VAL_-{PanelName}_-Panels.geojson	Any other panel set or validation target that is placed in the field but **not** used in the ELM. Replace {PanelName} with the target's name or unique identifier — e.g. a spare GRYFN 4-panel set becomes QC_VAL_Grfyn4P_Panels.geojson.
QC_GCP_-groundtruth_-points.geojson	**Reference** GCP locations measured independently in the field (e.g. Aeropoint, Trimble RTK). Here groundtruth is a **reserved target identifier** that flags this file as the surveyed reference layer — do not use it for any other purpose. Points-only file; one point per GCP, with an ID matching the field log.
QC_GCP_-points.geojson	**Observed** GCP locations as digitised from the drone orthomosaic in QGIS. Use this exact name (no target identifier) **only when every data product for the flight aligns perfectly** (i.e. one set of digitised points is valid for all orthomosaics). GCP_name must match the corresponding ID in QC_GCP_groundtruth_points.geojson so the two can be paired for residual reporting.
QC_GCP_-{Product}_-points.geojson	**Observed** GCP locations when the flight's data products **do not** all align to a single set of digitised points. Digitise a separate layer for each product and use the product name as the target identifier — e.g. QC_GCP_VNIR_points.geojson, QC_GCP_SWIR_points.geojson, QC_GCP_RGB_points.geojson. This naming is **strongly recommended**, as the accuracy report uses the identifier to label the per-product residuals. GCP_name must still match the corresponding ID in QC_GCP_groundtruth_points.geojson.
QC_-LIDAR_-{TargetName}_-Surface.geojson	Name for any future LiDAR calibration surfaces. Replace {TargetName} with the target identifier.

Worked example — most common field setup The most common field setup is **two GRYFN 4-panel sets**, the **GRYFN 2-panel validation set**, and a **set of GCPs**. The file names depend entirely on how the ELM was calculated in the GRYFN Processing Tool (GPT).

If a **single** GRYFN 4-panel set was used to create the ELM, the names are:

- QC_ELM_Panels.geojson — the 4-panel set used to create the ELM.
- QC_VAL_Grfyn4P_Panels.geojson — the 4-panel set **not** used in the ELM.
- QC_VAL_Panels.geojson — the GRYFN 2-panel validation set.
- QC_GCP_groundtruth_points.geojson — reference GCP locations measured independently in the field.
- QC_GCP_points.geojson — observed GCP locations digitised from the drone orthomosaic in QGIS.

If **both** GRYFN 4-panel sets were used together in the ELM, give each ELM file a target identifier. Here yellow and blue are the colours of the inventory tags attached to the panels — any other unique identifier (e.g. east / west) works just as well:

- QC_ELM_yellow_Panels.geojson — the first 4-panel set used in the ELM.
- QC_ELM_blue_Panels.geojson — the second 4-panel set used in the ELM.
- QC_VAL_Panels.geojson — the GRYFN 2-panel validation set.
- QC_GCP_groundtruth_points.geojson — reference GCP locations measured independently in the field.
- QC_GCP_points.geojson — observed GCP locations digitised from the drone orthomosaic in QGIS.

In either setup, if the flight's data products do **not** all align to a single set of digitised points, replace QC_-GCP_points.geojson with one QC_GCP_{Product}_points.geojson layer per product (e.g. QC_GCP_VNIR_-points.geojson, QC_GCP_SWIR_points.geojson) as described in the table above.

> **Tip**
>
> Any additional information (e.g. date) can be added to the end of the file name with an underscore — e.g. QC_-ELM_Panels_20260302.geojson. This table will be updated if we source panels other than GRYFN. Shapefile equivalents (e.g. QC_ELM_Panels.shp) are also accepted by the QA code.

File Storage

Any files created in the quality control steps are to be stored in a newly created QC_data folder inside T1_proc, following the APPN folder structure.

Formal path:

```
./{Node}/
 {YYYY_ProjectDesc[_I|E][_Researcher][_org]}/
 {YYYYSiteName[_F|C]}/
 {SensorPlatform}/{YYYYMMDD}/run_XX/T1_proc/QC_data/
```

Example:

`./USYD_Narrabri/2025_SIFCal/2025IAWatson_F/CALVIS/20250825/run_00/T1_proc/QC_data/`

These vector files (GeoJSON preferred, shapefile accepted) are meant to be created after the GPRO has been completed. The APPN storage repo has been updated to include this folder: https://github.com/ArdenB/APPN_GenricFileStorage

---

Positional QC

> **Warning**
>
> This section is still a work in progress.

> **Important**
>
> Positional QC **must be performed immediately after processing is complete** (e.g. straight after GPRO creation). Positional errors are often the first indication of a larger problem with the flight or the processing pipeline, and almost always require the data to be reprocessed before they can be resolved. Catching them early avoids wasted downstream work.

APPN Positional QC is performed in three stages:

1. **Field Data Collection**
2. **Observed point capture**
3. **Accuracy reporting**

   The descriptions below show the procedure for the CALViS using GCPs. The process for the GOBI is the same, but there is only the VNIR orthomosaic. The same workflow can be used to produce shapefiles if preferred — just choose *ESRI Shapefile* in place of *GeoJSON* in the format dropdown.

For the CALViS, in the products folder of the completed GPRO you will find the `.tif` files:

- `X__VNIR_Orthomosaic.rgb`
- `X_SWIR_Orthomosaic.rgb`

These are the RGB bands from the VNIR and SWIR files respectively. They can be easier to use than the full `.bin` files, though the procedure is the same.

Field Data Collection

Independent GCPs are placed in the field during data capture (see Standard Flight and Validation Flight). These GCPs are independent of any points used during processing so they provide an unbiased reference ("ground truth") to compare the drone-derived positions against.

The **raw** data exported from the GCP hardware (Aeropoint CSVs, Trimble job files, etc.) should be saved to the `T0_raw/Vault` folder so the original measurements are preserved.

Formal path:

```
./{Node}/
{YYYY_ProjectDesc[_I|E][_Researcher][_org]}/
{YYYYSiteName[_F|C]}/
{SensorPlatform}/{YYYYMMDD}/run_XX/T0_raw/Vault/
```

Example:

`./USYD_Narrabri/2025_SIFCal/2025IAWatson/CALVIS/20250825/run_00/T0_raw/Vault/`

The raw data must then be converted into the APPN standard reference format: a GeoJSON with one point per GCP and the fields `ID`, `X`, `Y`, `Z`, plus an explicit CRS. The exact conversion process depends on the GCP hardware (Aeropoint, Trimble, etc.).

> **Note**
>
> A step-by-step conversion process for each supported GCP type (Aeropoint, Trimble, …) will be added in a future revision. This item is tracked in the future-release backlog. Until then, follow the hardware vendor's export workflow and the field-checks listed below.

Common issues to watch for during conversion:

- Mismatched CRS between GCP collection and data processing.
- Incorrect height datum — e.g. GDA2020 ellipsoidal heights vs the Australian Height Datum (AHD). See Geoscience Australia's AUSGeoid2020 conversion tool for converting between the two.
- Inconsistent `ID` names or duplicate points — fix or remove before saving.

Once the issues are resolved, save the cleaned reference points as `QC_GCP_groundtruth_points.geojson` in the `T1_proc/QC_data/` folder (see Naming Conventions and File Storage).

Formal path:

```
./{Node}/
{YYYY_ProjectDesc[_I|E][_Researcher][_org]}/
{YYYYSiteName[_F|C]}/
{SensorPlatform}/{YYYYMMDD}/run_XX/T1_proc/QC_data/QC_GCP_groundtruth_points.geojson
```

Example:

```
./USYD_Narrabri/2025_SIFCal/2025IAWatson/CALVIS/20250825/run_00/T1_proc/QC_data/QC_GCP_groundtruth_points.g
```

Observed point capture

Manually digitise a matched set of points from the drone orthomosaic in QGIS and save them as `QC_GCP_points.geojson` (see Naming Conventions). The `GCP_name` column **must** match the `ID` values used in `QC_GCP_groundtruth_points.geojson` so that each observed point can be paired with its reference for accuracy reporting.

1. Load the Data

1. Load the SWIR and VNIR files into QGIS and run the following checks:

- Do the panels in the SWIR and VNIR overlap?
- Make note of the CRS.

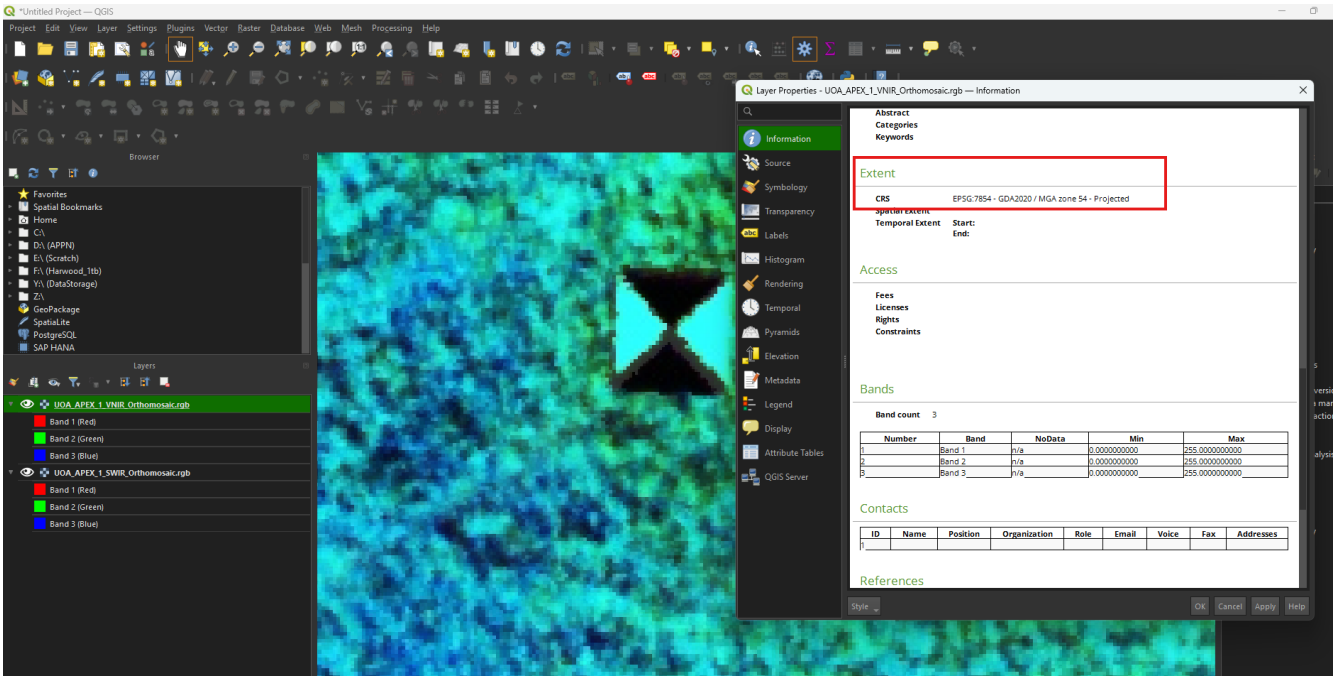


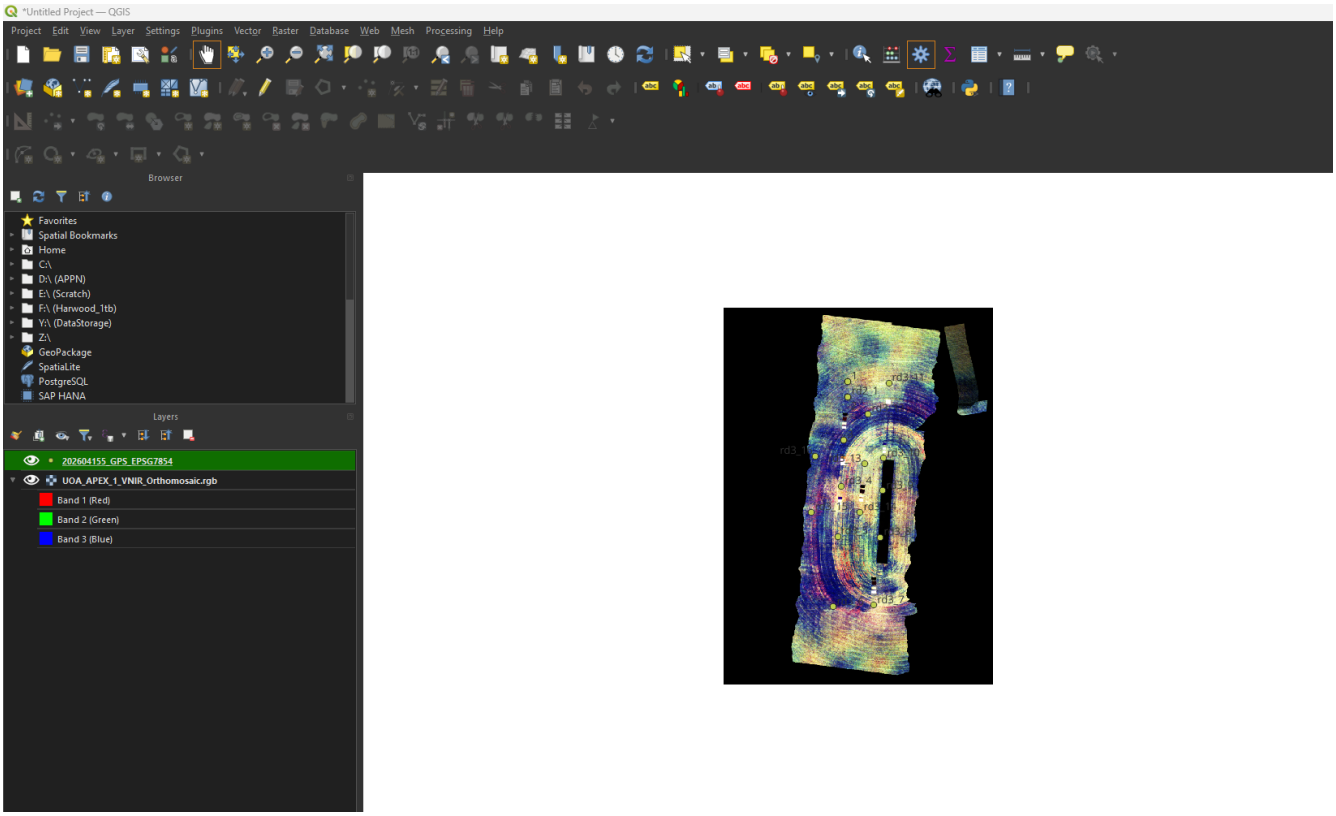
Figure: The VNIR is set to 50% opacity to confirm the panels overlap with the SWIR. The information panel is open on the SWIR to confirm the CRS (EPSG:7855 – GDA2020 / MGA zone 54). This CRS is for Roseworthy SA where this CALViS flight was collected.

> **Caution**
>
> For CALViS, the SWIR and VNIR orthomosaics **must** be checked independently. The two sensors are georeferenced separately, and we have seen flights where the GNSS correction was applied cleanly to one orthomosaic but was broken or offset in the other. Confirming each one on its own is the only reliable way to catch this.

Instead of transparency you can also toggle one image on and off to confirm the overlap is acceptable.

If overlap is fine:

2. Load the ground-truth GPS data (`QC_GCP_groundtruth_points.geojson`) for your GCPs. The reference layer is loaded first so it is easy to name the digitised points on the drone imagery to match.



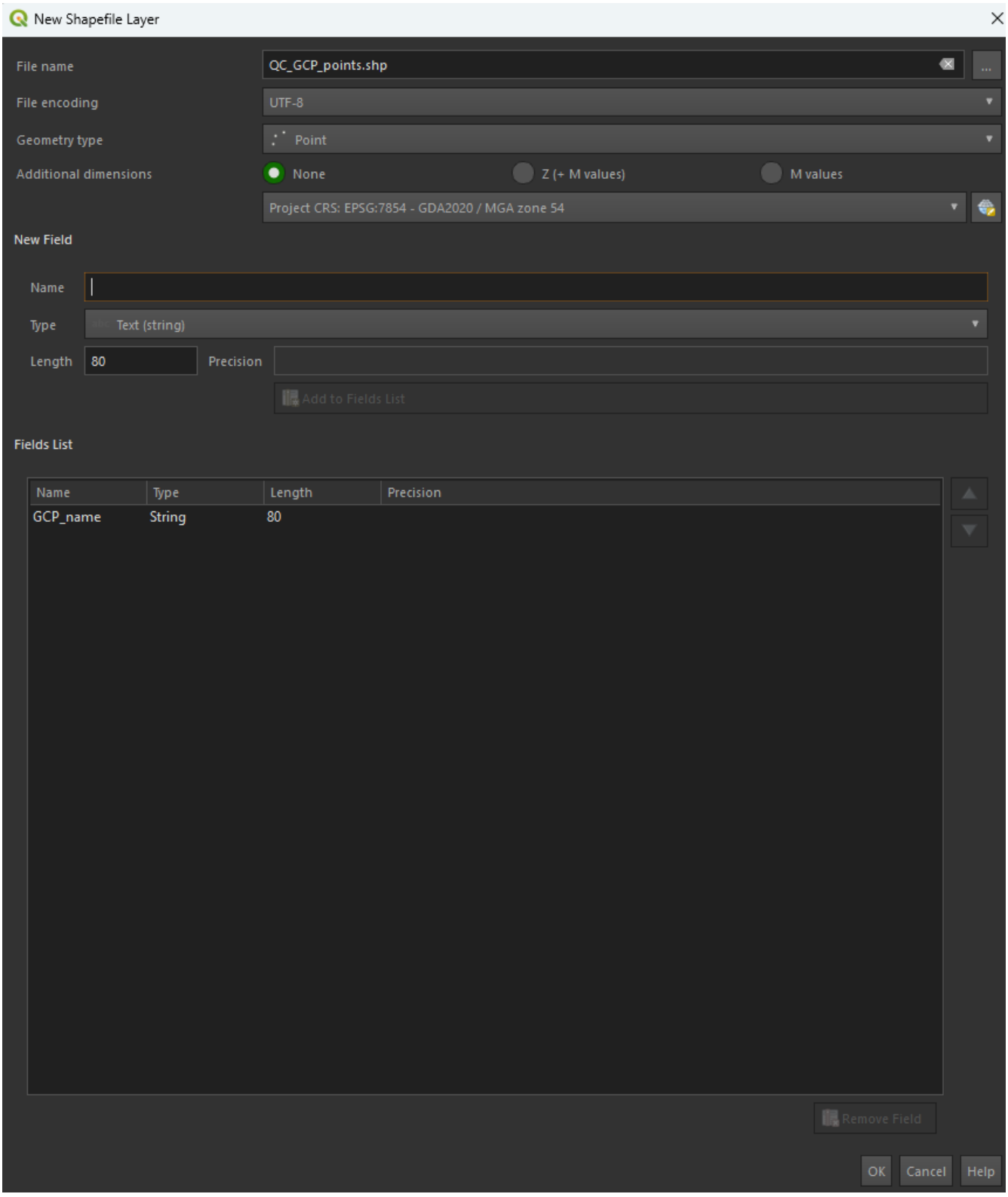
Example of the ground-truth points over a GCP visible in the CALViS orthomosaic:



2. Create the Vector Layer

1. Navigate to **Layer** → **Create Layer** → **New Shapefile Layer**.

Set the **File Name** to `QC_GCP_points`, the **Geometry type** to *Point*, and replace the pre-filled `id` field with `GCP_name` of type **Text (string)**.

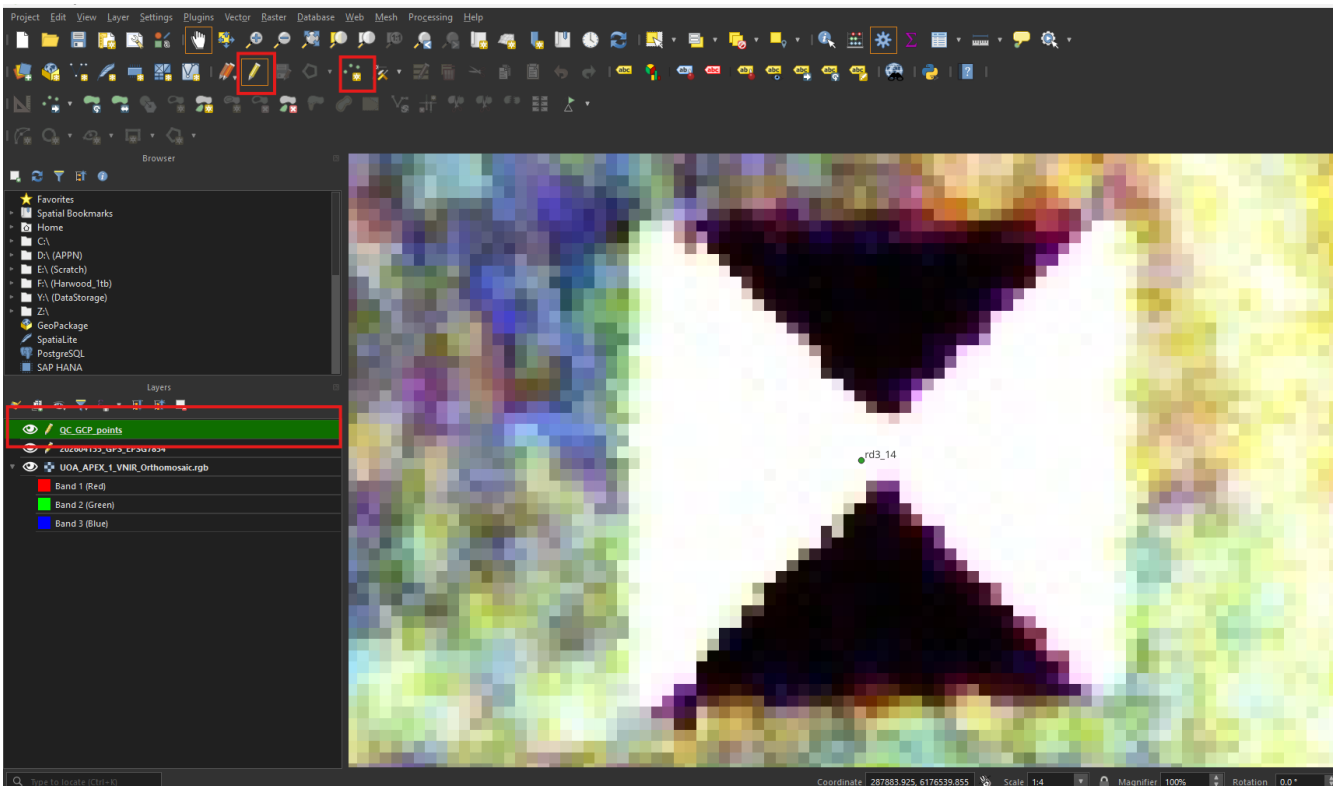


2. Set the **File Name** to `QC_GCP_points.shp` (it will be exported to GeoJSON in step 5).
3. Set the **Geometry type** to *Point*.
4. Set the **CRS** to match the orthomosaic.
5. Add a single field — `GCP_name`:

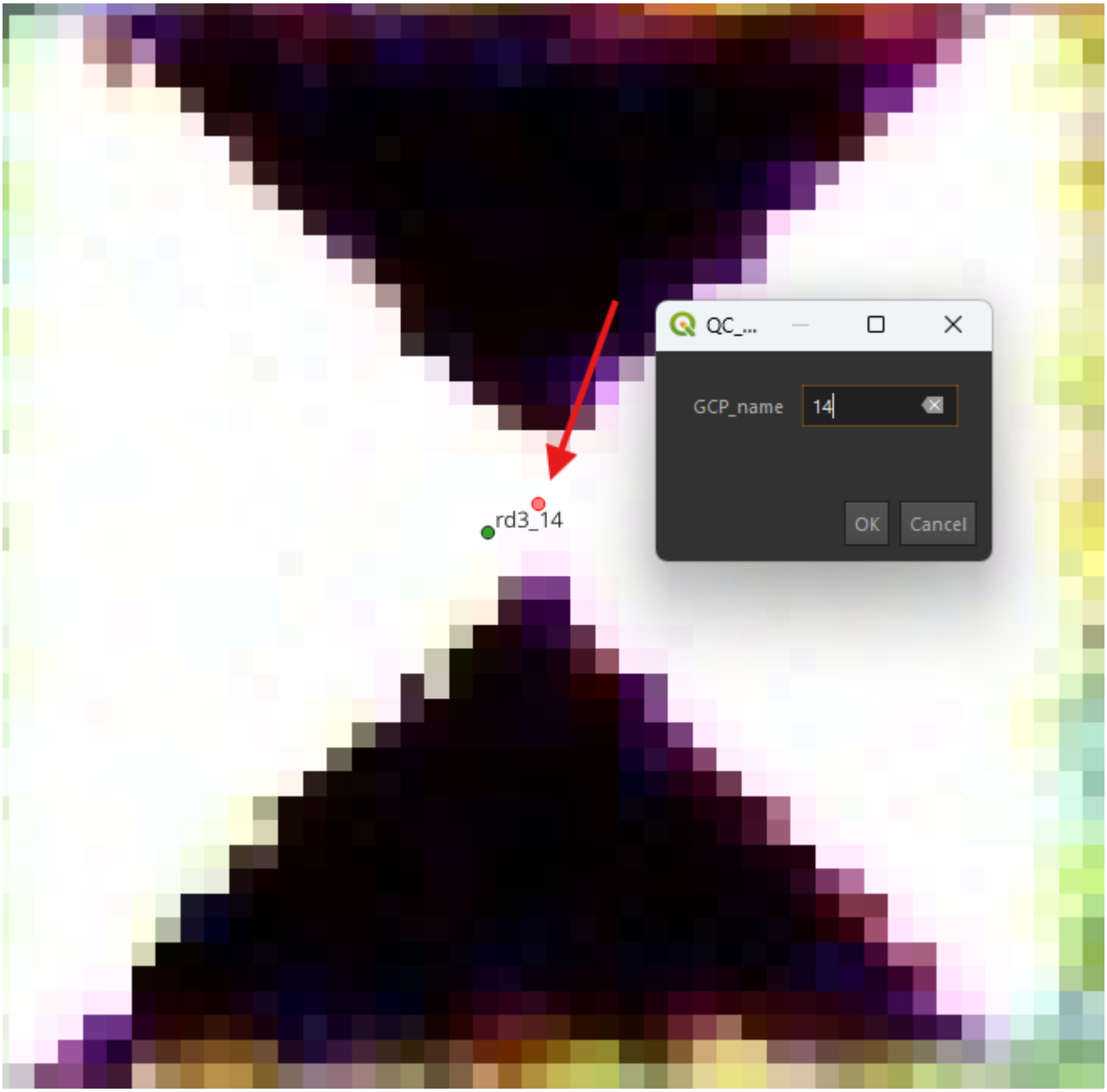
- Select the pre-filled `id` field in the Fields list and click **Remove Field** (bottom right) to remove it.
- Add `GCP_name` as **Text (string)**.

3. Annotating the GCPs To start annotating the drone orthomosaic:

1. Select `QC_GCP_points` in the **Layers** menu.
2. Click the pencil icon (**Toggle Editing**).
3. Click **Add Point Feature**.



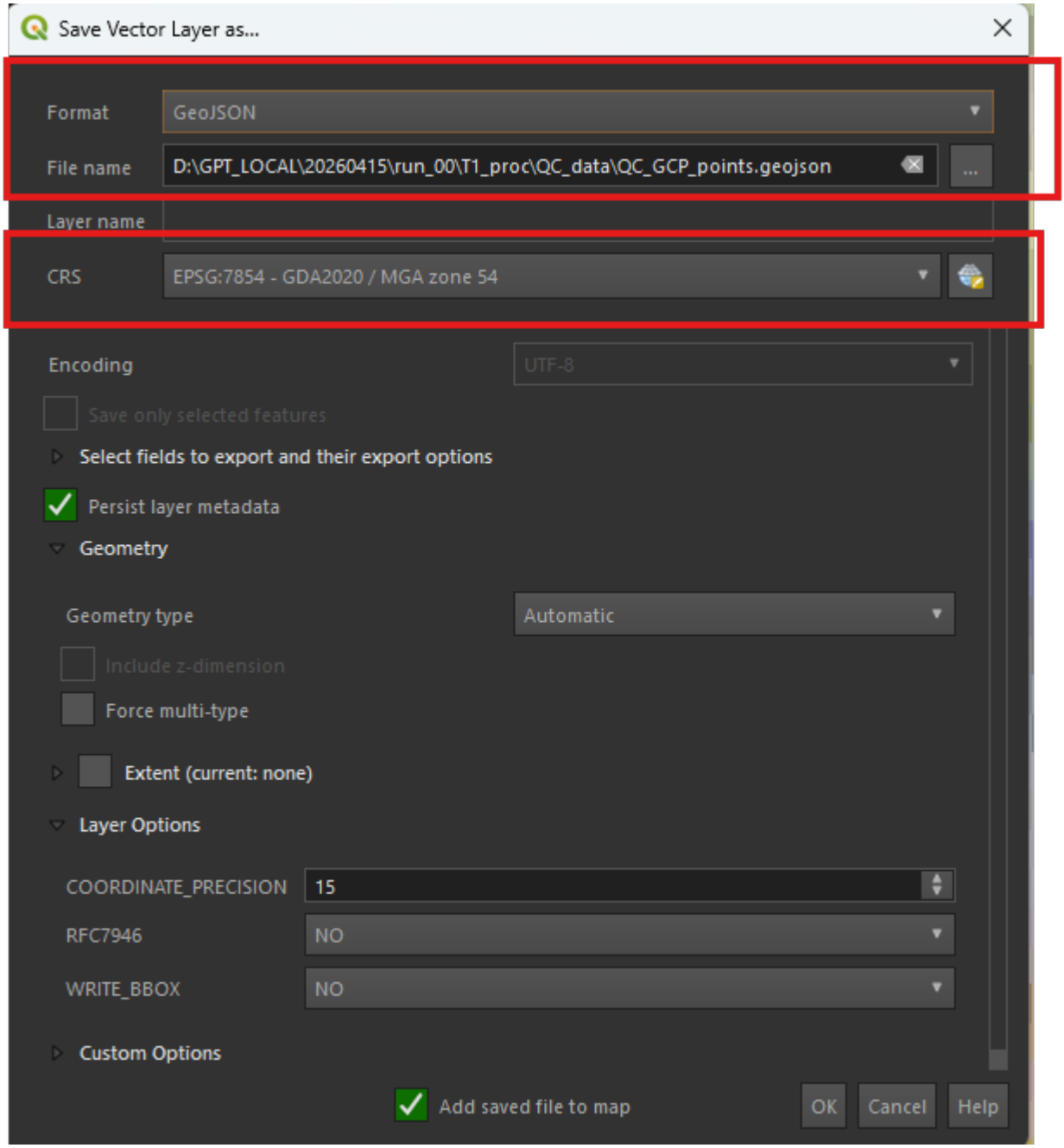
4. Navigate to the centre of each GCP, left-click to drop a point, and enter the `GCP_name` so it matches the corresponding `ID` in `QC_GCP_groundtruth_points.geojson`. In the figure below the ground-truth point is green and the digitised (observed) point is red — the offset between them is the positional error being measured.



Repeat for every GCP visible in the orthomosaic.

5. Save the data as a GeoJSON

1. Right-click `QC_GCP_points` in the **Layers** menu → **Export** → **Save Features As...**.
2. Set **Format** to *GeoJSON*, confirm the **File Name** and target folder (click the three dots to navigate), and double-check the **CRS** matches the orthomosaic.



Accuracy reporting

Run the QA code at https://github.com/ArdenB/APPN_GenricFileStorage (`Code/DS02_DatasetQA/`) to generate a spatial accuracy report comparing the observed (drone) and reference (surveyed) GCP locations. The report produces per-point residuals and overall RMSE in X, Y, and Z.

The code pairs each observed point in `QC_GCP_points.geojson` with the matching reference point in `QC_GCP_groundtruth_points.geojson` using the `GCP_name` / `ID` fields (see Naming Conventions), so those identifiers **must** match exactly for a point to appear in the report.

What the report contains

- **Per-point residuals** — the signed difference between the observed and reference position for every paired GCP, in metres, broken out as `dX`, `dY`, and `dZ` (easting, northing, and height) plus the horizontal (`dXY = sqrt(dX² + dY²)`) and total 3D offset. These identify whether an error is uniform across the site (suggesting a systematic shift or datum problem) or localised to individual points (suggesting a digitising or ground-truth

error).

- **Summary statistics** — overall **RMSE** in X, Y, and Z, the horizontal and 3D RMSE, and the mean (bias) and standard deviation of the residuals. A large mean relative to the standard deviation indicates a systematic offset; a large standard deviation indicates noisy or inconsistent georeferencing.
- **Point count** — the number of GCPs successfully paired. If this is lower than the number of GCPs placed in the field, check for unmatched `GCP_name` / `ID` values before interpreting the result.

How to interpret the result

1. Check the **point count** first — a report built from only one or two paired GCPs is not a reliable accuracy estimate.
2. Inspect the **mean residual (bias)**. A consistent non-zero bias in X, Y, or Z across all points usually points to a CRS / height-datum mismatch (see the common issues under Field Data Collection) rather than a true positional error, and can often be corrected without reprocessing.
3. Inspect **individual large residuals**. A single GCP with a much larger offset than the rest is most often a mis-digitised observed point or a bad ground-truth measurement — re-check that point before concluding the flight has failed.
4. Compare the **RMSE** against the acceptance thresholds below.

Where to save the report

Save the generated report alongside the input vector layers in the `T1_proc/QC_data/` folder (see File Storage) so the residuals travel with the dataset.

> **Important**
>
> **Pass / fail thresholds.** Flag a flight for **caution** when the RMSE exceeds **5 cm**, and treat it as a **fail** when the RMSE exceeds **10 cm**. Record the reported values in the QC log and review any flight that breaches the caution threshold. These limits will be evaluated further during the APEx flights to confirm they are appropriate.

Spectral QC

Spectral QC is done by drawing polygons in GIS software over surfaces with known spectral values, then extracting and comparing them. This is done after all processing in software like GPT is complete.

CALViS and GOBI flights conducted prior to the *Operational Excellence in APPN Hyperspectral Imaging* SIF likely consist of one GRYFN reflectance panel (used to generate the ELM in the GRYFN Processing Tool) along with additional panels for validation.

File naming, format (GeoJSON preferred), and storage location for the polygons created below all follow the General QC Conventions above.

Creating Geospatial Vector Data — GOBI & CALViS (QGIS)

> The description below shows the procedure for creating GeoJSON files for the CALViS using GRYFN reflectance panels for ELM. The process for the GOBI is the same, but there is only the VNIR orthomosaic. The same workflow can be used to produce shapefiles if preferred — just choose *ESRI Shapefile* in place of *GeoJSON* in the format dropdown.

For the CALViS, in the products folder of the completed GPRO you will find the `.tif` files:

- `X__VNIR_Orthomosaic.rgb`
- `X_calvis_sif_cal_20250925_SWIR_Orthomosaic.rgb`

These are the RGB bands from the VNIR and SWIR files respectively. They can be easier to use than the full `.bin` files, though the procedure is the same.

1. Load the Data

1. Load both files into QGIS and run the following checks:

- Do the panels in the SWIR and VNIR overlap?
- Make note of the CRS.

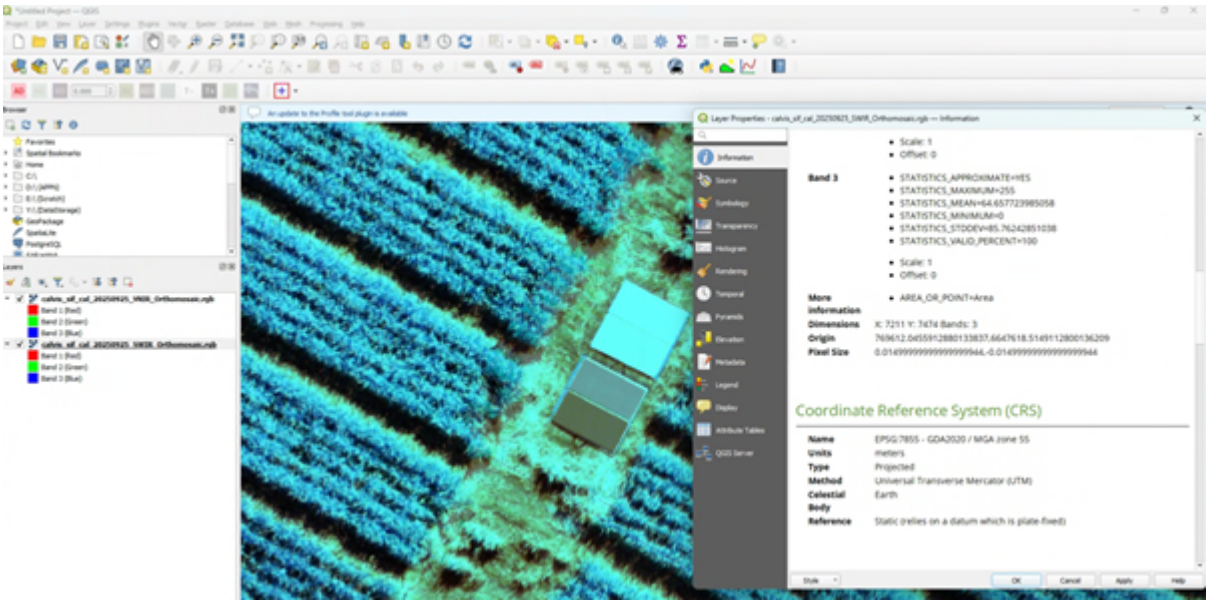
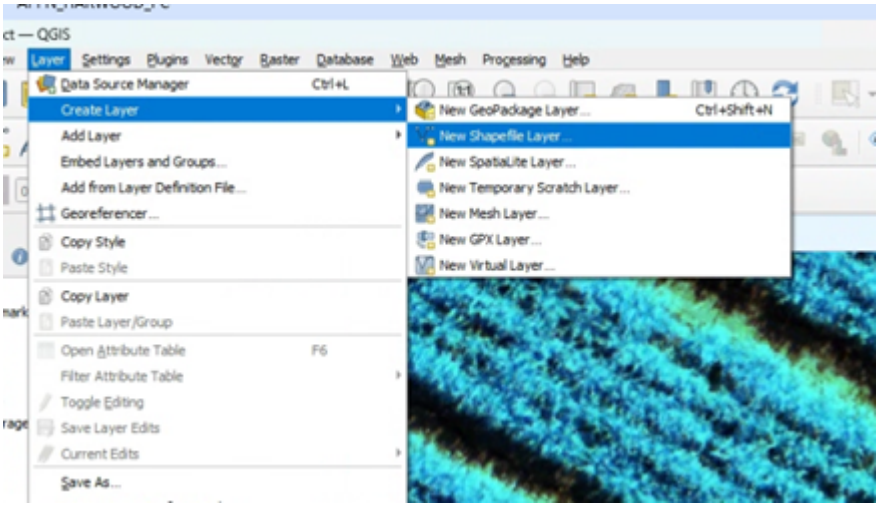


Figure: The VNIR is set to 50% opacity to confirm the panels overlap with the SWIR. The information panel is open on the SWIR to confirm the CRS (EPSG:7855 – GDA2020 / MGA zone 55). This CRS is for Narrabri NSW where this CALViS flight was collected.

2. Create the Vector Layer Note

This section is pending review and expansion (panel-specific guidance) by Richard Harwood, tracked in the future-release backlog.

1. Navigate to **Layer** → **Create Layer** → **New GeoPackage Layer...** for GeoJSON output, or **New Shapefile Layer...** if producing a shapefile. Either way you will set the output **File Name** with the appropriate extension (`.geojson` preferred, `.shp` accepted).



Tip

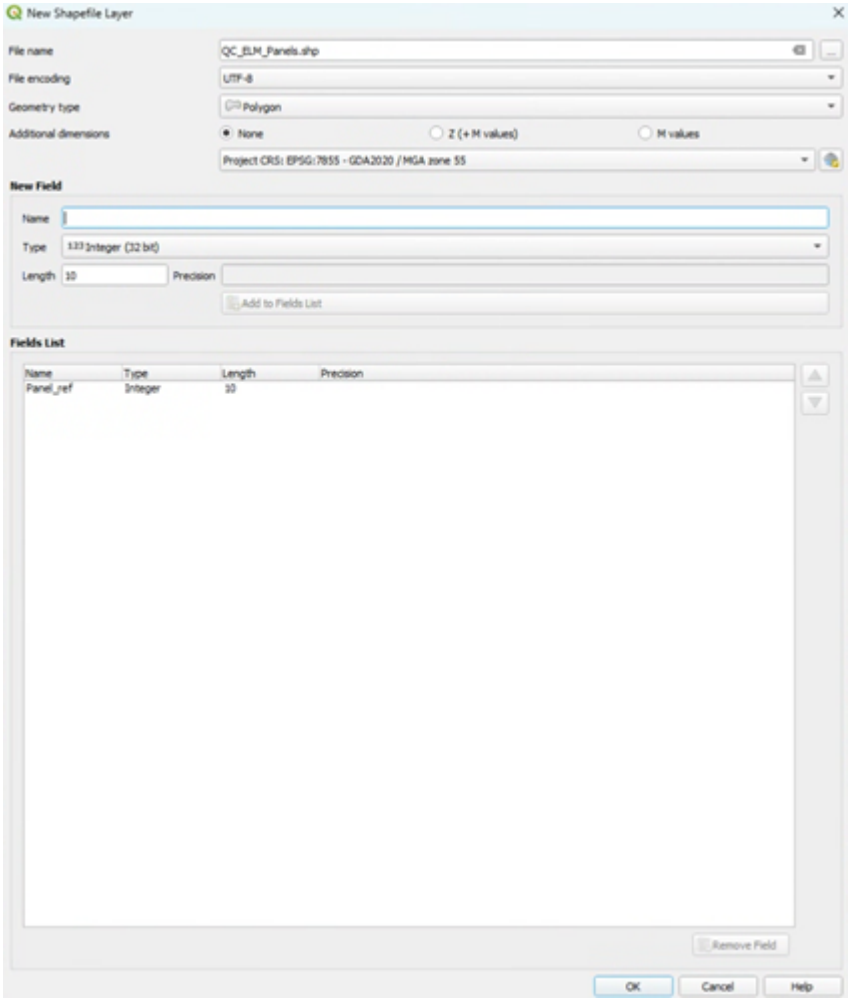
To save directly as GeoJSON you can also create a temporary scratch layer and then **Export** → **Save Features As...** → **GeoJSON** in step 5.

2. Set the **File Name** to `QC_ELM_Panels.geojson` (see the naming conventions table for the correct name given

your use case).
3. Set the **Geometry type** to *Polygon*.
4. Set the **CRS** to match your dataset.
5. Add a single field — `Panel_ref`:

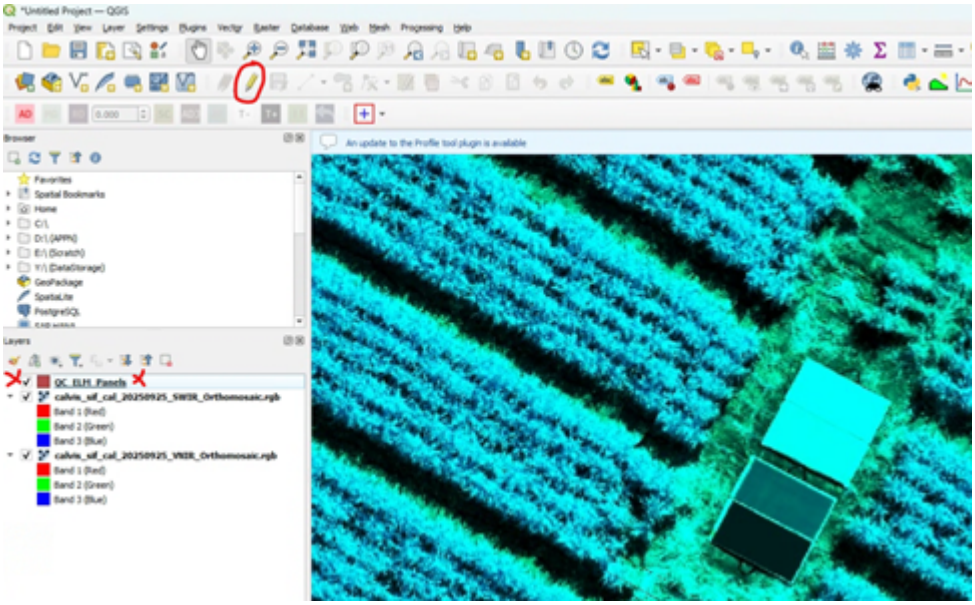
- Select the pre-filled `id` field in the Fields list and click **Remove Field** (bottom right) to remove it.
- Set `Panel_ref` as an **Integer** between 0–100. It is the percentage reflectance (e.g. 11, 30, 56, 82 for GRYFN panels).

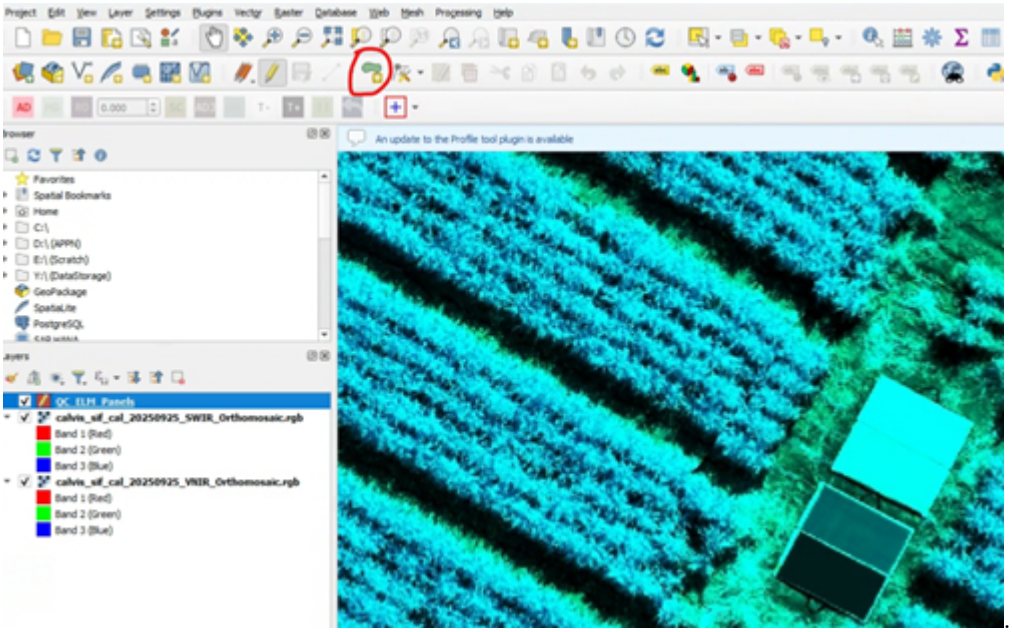
> **Important**
>
> You need to click **Add to Fields List** once you populate "New Field".



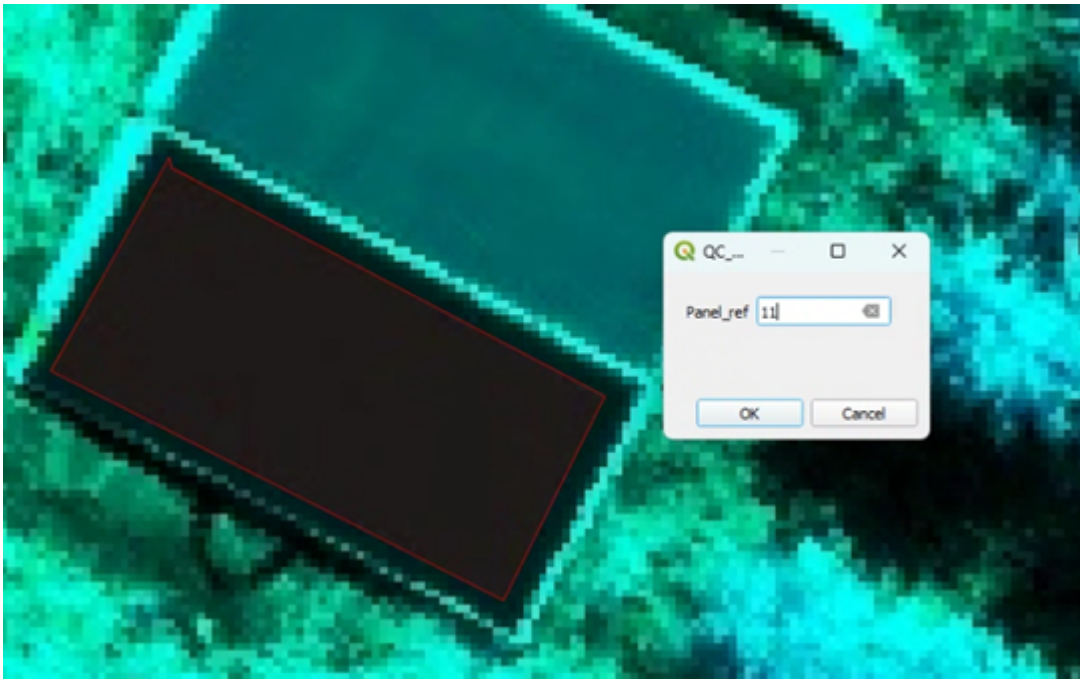
3. Draw Boxes over the Panels The boxes should be polygons drawn inside the panel. The points should be drawn to cover as much of the panel as possible without including edge effects (2–3 pixels from the edge of the panel). If there are gaps or missing values within the panel, they should be included.

1. Select `QC_ELM_Panels` in the **Layers** menu.
2. Click the pencil icon (**Toggle Editing**).



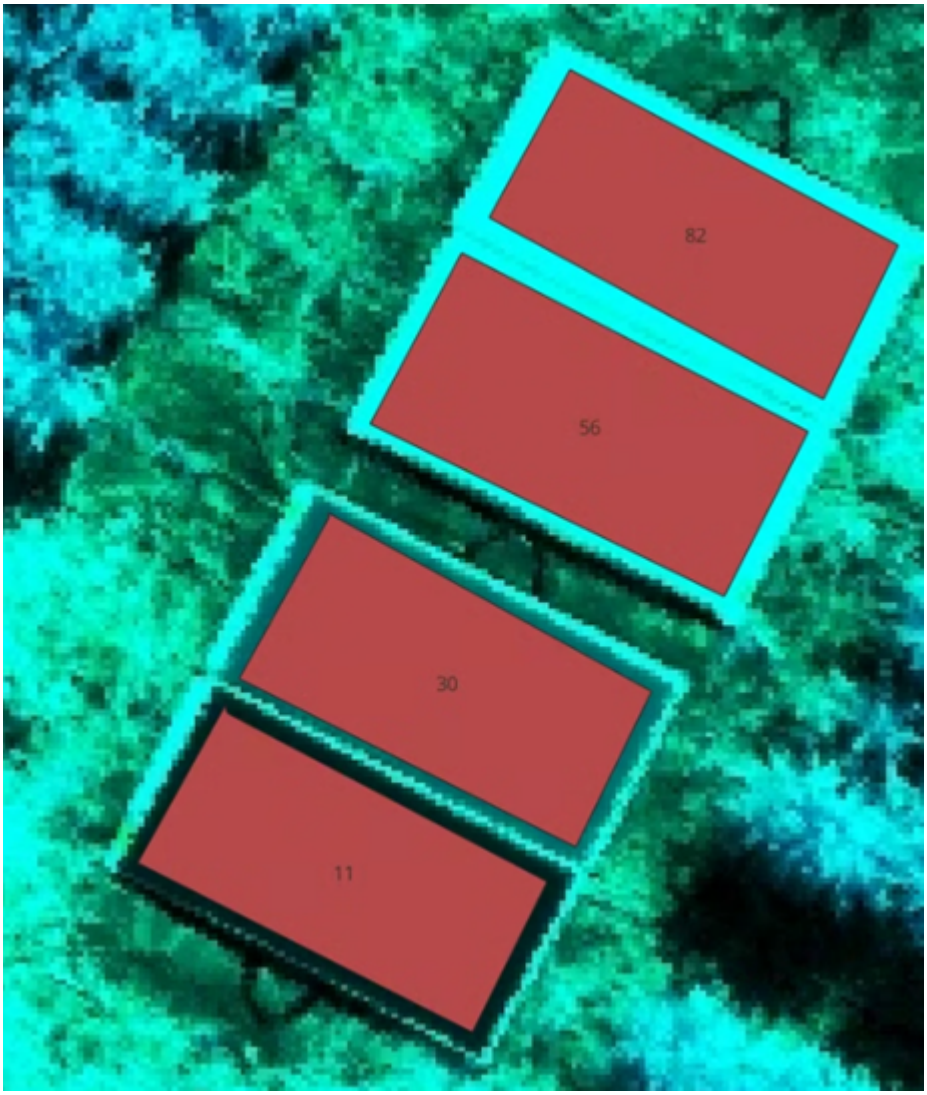


3. Click **Add Polygon Feature**
4. Click four points around a specific reflectance panel, then right-click to close the polygon. Repeat for all panels (e.g. 11, 30, 56, 82).



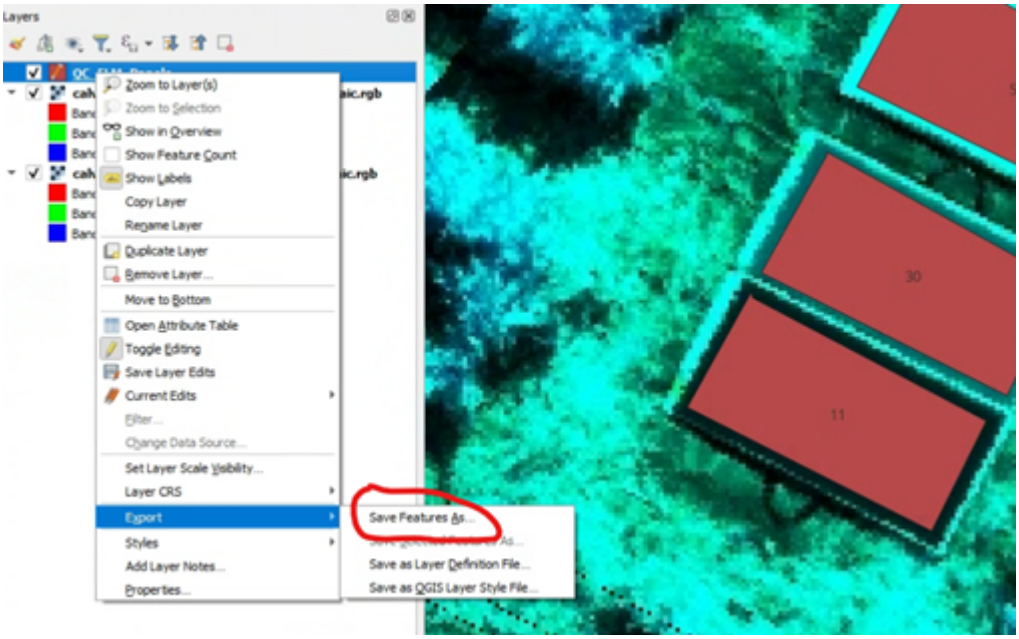
4. Sanity Check Labels

1. Double-click `QC_ELM_Panels` in the **Layers** menu.
2. In **Layer Properties**, click **Labels** and select **Single Labels**.

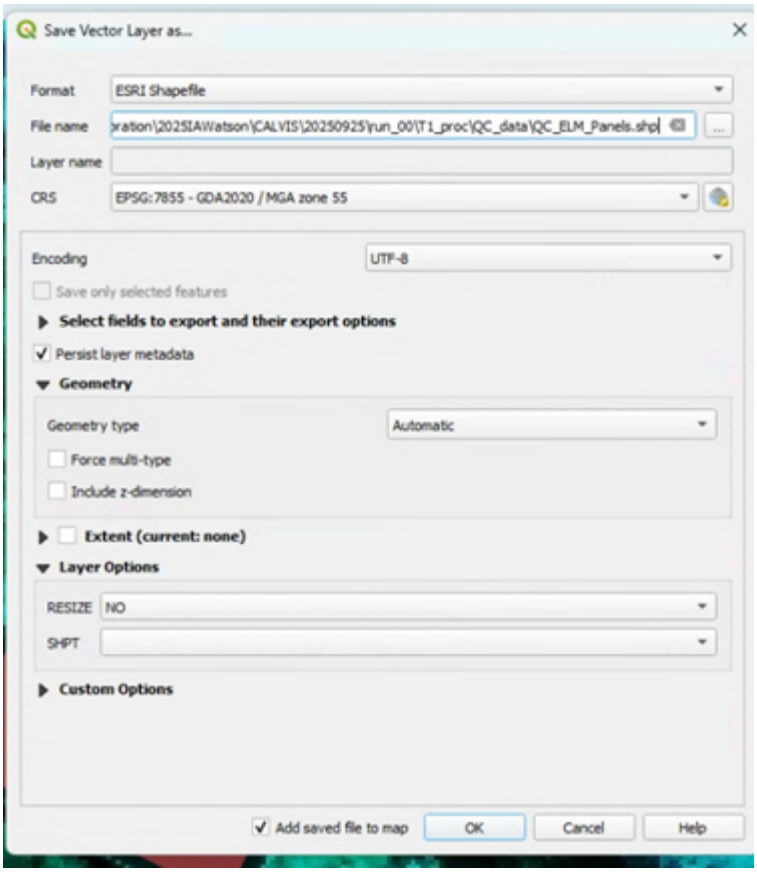


5. Save the File

1. Right-click `QC_ELM_Panels` in the **Layers** menu → **Export** → **Save Features As**.



2. Set the **Format** to *GeoJSON* (preferred) — or *ESRI Shapefile* if required. Make sure the **File Name** is correct and in the right folder (click the three dots to navigate). Double-check the **CRS**.



Extracting Pixels into a Table

Arden Burrell has made a Python script that can go through the APPN standard folder structure, extract the values into a table, and save that as a `.csv` or `.parquet` file automatically. The code is available from:

https://github.com/ArdenB/APPN_GenricFileStorage

- **Script:** `Code/DS02_DatasetQA/QA00_SpectralValidation.py` — per-run extraction, QC report (`QC_data/QC_spectra_report.json`), and per-run spectra figure.
- **README:** `Code/DS02_DatasetQA/README.md`

The extracted tables are written to a `QC_Spectral_Tables/` folder inside the run's `QC_data/` and are named after the panel file they came from:

`QC_{ELM|VAL}[_{TargetIdentifier}]_spectra_{VNIR|SWIR}[_gproN][_{Extra}].{parquet|csv}`

e.g. `QC_ELM_spectra_VNIR.parquet` or `QC_VAL_Gryfn4P_spectra_SWIR.parquet`.

If nodes choose to extract the points using other means, the tables should have the following columns:

`band, wavelength, value, Panel_ref, node, project, site, sensor, date, run, panel_name, EM_Region, gpro_nu`

where `EM_Region` is the electromagnetic region of the source orthomosaic (`VNIR` or `SWIR`). The QA00 script additionally records `target_type` (`ELM`/`VAL`), `target_id` (the optional target identifier), `panel_set` (the physical panel set identified from the `Panel_ref` signature, e.g. `Gryfn4P` = {11, 30, 56, 82}, `Gryfn2P` = {20, 45}) and a boolean `Valid_Range` QC flag.

Comparing Runs (multi-run QC)

Cross-run comparison figures (per-panel reflectance and residual spectra, one line per run) are produced by a second script in the same repository:

- **Script:** `Code/DS02_DatasetQA/QA02_SpectralRunComparison.py`

It never opens the orthomosaics — it gathers the extracted `QC_Spectral_Tables` produced by QA00 across every run under the given path. Typical usage:

```
# Extract per-run tables + reports, then compare every run under a node
python Code/DS02_DatasetQA/QA00_SpectralValidation.py --path /path/to/Node
python Code/DS02_DatasetQA/QA02_SpectralRunComparison.py --path /path/to/Node
```

- Figures are saved to `<Node>/Documents/QCReports/` (node path) or `<Project>/Documentation/QCReports/` (project path).
- `--start-date` / `--end-date` limit which runs are compared (inclusive).
- `--save-dir` writes copies of every gathered table to one folder for sharing with other nodes (filesender/globus); receiving nodes combine them with their own data via `--load-dir`.

Accuracy reporting

> **Note**
>
> A spectral accuracy-reporting procedure is planned but not yet defined. Developing an acceptable spectral accuracy baseline requires data from the APEx flights; once that data is available, this section will document how the extracted panel reflectances are compared against their known values and the pass/fail criteria that result. This item is tracked in the future-release backlog.

LiDAR QC

> **Warning**
>
> This section is still a work in progress.

> **Note**
>
> Both the LiDAR **sampling** and **accuracy-reporting** procedures are planned but not yet defined. Developing the sampling workflow and an acceptable LiDAR accuracy baseline requires data from the APEx flights; once that data is available, this section will document how LiDAR calibration surfaces are sampled, how the results

are compared against their reference values, and the pass/fail criteria that result. This item is tracked in the future-release backlog.